P1 - MCP Security Gateway: Project Theory

Oct 10, 2026 · @Mayank

1. Idea and problem statement

The MCP Security Gateway is a checkpoint that sits between AI agents and the tools they use, and checks every tool call before it runs.

Background

- MCP (Model Context Protocol) is the standard way an AI agent connects to tools: files, databases, email, GitHub, and more.
- Today an agent usually connects directly to each MCP server and is trusted to use it responsibly.

Problem statement

- An agent can be tricked by hidden instructions inside a document, a webpage, an email, or even a tool's own description. This is called prompt injection or tool poisoning.
- A tricked agent can delete files, leak data, or send messages, and nothing stands in its way.
- Teams also cannot answer basic questions: which agent called which tool, with what input, and was it allowed?
- Tool descriptions can change silently after a user approves them, so a safe tool can turn unsafe later.

Who needs this

- Companies deploying AI agents that touch real systems.
- Developers building agents who want safety without rewriting each tool.
- Security teams who need an audit trail of agent actions.

2. What it can do and the solution

The solution is a proxy server: the agent connects to the gateway, and the gateway connects to the real MCP servers.

What it can do

- Allow or block tool calls per agent (for example, "this agent may only read files under `/docs`").
- Check the arguments of every call against a schema and rules.

- Limit how often an agent can call a tool.
- Raise an alert when a tool's name or description changes.
- Pause risky actions and ask a human to approve.
- Scan tool results for hidden instructions before the agent sees them.
- Write a complete log of every call and decision.

What we build

- A proxy that speaks MCP on both sides, so existing agents and tools work unchanged.
- A policy file (rules written as code) that says who may do what.
- An audit log stored in a database.
- A set of attack tests and benchmark scripts to prove it works.

Why this is different from "just another wrapper"

- It enforces rules outside the model, so a tricked model still cannot break them.
- The model cannot talk its way past the gateway, because the gateway does not use the model to decide.

3. Theory: the concepts behind it

Seven ideas explain how this project works; each is described below in plain words.

Concept	Plain meaning	Why it matters here
MCP	A standard way for AI apps to call tools, using JSON messages (JSON-RPC) over stdio or HTTP	The gateway must read and forward these messages without breaking them
Proxy	A middle server that passes traffic between two sides	Lets us add checks without changing agents or tools
Authorization	Deciding who may do what	Each agent gets its own allowed tools and arguments
Policy engine	Rules stored as code or config, evaluated on each request	Keeps rules separate from the gateway code and easy to test
Prompt injection	Hidden text that makes a model follow an attacker's instructions	The main threat the gateway defends against
Tool poisoning	A malicious or changed tool description that steers the agent	Fixed by hashing tool definitions and alerting on change
Rate limiting	Capping calls per time window	Stops runaway loops and abuse

Key theory points

- **Defense outside the model:** a model cannot be fully trusted to refuse attacks, so rules must be enforced by normal code.
- **Least privilege:** each agent gets only the tools and arguments it needs, nothing more.
- **Fail-closed vs fail-open:** if the policy store is down, a fail-closed gateway blocks calls (safe, but stops work); a fail-open gateway allows them (works, but unsafe). We choose one and document it.
- **Streaming trade-off:** checking a result needs the full result, but waiting for it adds delay. We decide between buffering and streaming.
- **Hash for change detection:** a hash is a fingerprint of text. If a tool description changes, its fingerprint changes, so we can detect it.
- **Heuristics have errors:** injection detection will miss some attacks (false negatives) and flag some safe text (false positives), so we measure both.

Existing work to study (check current state before claiming any gap)

- The MCP specification and its security guidance.
- Existing MCP proxies and gateways.
- OWASP guidance on LLM application risks.

4. How it works, stack, challenges, metrics, roadmap

Every tool call follows one path through the gateway, and each step can stop it.

Request flow

1. The agent sends a tool call to the gateway.
2. The gateway identifies the agent and loads its policy.
3. It checks the tool name, then the arguments against the schema and rules.
4. It checks the rate limit.
5. Risky calls pause for human approval.
6. Allowed calls are forwarded to the real MCP server.
7. The result is scanned for hidden instructions.
8. The result is returned to the agent, and the whole event is written to the audit log.

Tech stack (start simple)

Part	Choice	Why
Gateway	Python (FastAPI) first; Go or Rust later if speed matters	Fast to build; you already use FastAPI

Part	Choice	Why
Policy rules	YAML/JSON config, then a small rule engine	Easy to read and test
Storage	Postgres for policies and logs	Reliable, queryable audit trail
Rate limits	Redis	Fast counters
Observability	OpenTelemetry and Prometheus	Standard tracing and metrics
Packaging	Docker Compose	One-command demo

Hard problems

Problem	Why it is hard	How to test
Protocol fidelity	The proxy must not break MCP behavior	Run a real MCP client test suite through it
Added latency	Every check adds delay	Measure p50/p95/p99 with and without the gateway
Streaming results	You cannot inspect what you have not received	Compare buffered vs streamed designs
Policy correctness	One bug lets a bad call through	Property tests: a denied call never reaches the server
Tool poisoning	Descriptions can change silently	Build a malicious test server that changes its description
Injection in results	Detection has false alarms	Measure precision and recall on a labeled attack set
Concurrency	Many agents at once	Load test and watch connection limits
Store outage	Policy database may go down	Kill it during a test and check the chosen behavior

Metrics to report (measure them; do not invent them)

- Added latency percentiles at a stated request rate.
- Maximum requests per second before degradation.
- Attack detection rate and false-positive rate on your attack set.
- Policy evaluation time per call.
- Always note the hardware and test setup used.

Roadmap (about 8-10 weeks, one developer)

1. Weeks 1-2: study the MCP spec and build a pass-through proxy.
2. Weeks 3-4: add allowlists, schema checks, and logging.
3. Weeks 5-6: add the policy engine, rate limits, and approval hook.
4. Weeks 7-8: write attack tests, load tests, and benchmarks.
5. Weeks 9-10: add tracing, Docker packaging, documentation, and a demo video.